Assignment 9: Advanced SQL Queries Using Joins and Subqueries

Hotel reports using joins, grouping and subqueries

Project Topic: Hotel Management System

1. Objective

Advanced queries combine related tables and use subqueries to produce useful reports for hotel staff and management. The examples use the sample data from Assignment 7.

2. Report 1: Reservations with Guest and Room Details (JOIN)

```
SELECT r.reservation_id, CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
       rm.room_number, r.check_in_date, r.check_out_date, r.booking_status
FROM Reservations r
JOIN Guests g ON r.guest_id = g.guest_id
JOIN Rooms rm ON r.room_id = rm.room_id
ORDER BY r.reservation_id;
```

Purpose: Shows each reservation with the guest name and room number. Expected output has five rows: 1001 Asha Rao room 102; 1002 Rahul Shah room 201; 1003 Meera Nair room 202; 1004 Arjun Das room 101; 1005 Sara Khan room 102.

3. Report 2: Reservation Payment Summary (LEFT JOIN and GROUP BY)

```
SELECT r.reservation_id, CONCAT(g.first_name, ' ', g.last_name) AS guest_name,
       COALESCE(SUM(p.amount), 0) AS recorded_payment_total,
       SUM(CASE WHEN p.payment_status = 'Paid' THEN p.amount ELSE 0 END) AS paid_amount
FROM Reservations r
JOIN Guests g ON r.guest_id = g.guest_id
LEFT JOIN Payments p ON r.reservation_id = p.reservation_id
GROUP BY r.reservation_id, g.first_name, g.last_name
ORDER BY r.reservation_id;
```

Purpose: Summarizes recorded payment amounts for each reservation and separates amounts marked Paid. Expected recorded totals: 1001 = 6400; 1002 = 6400; 1003 = 13000; 1004 = 1800; 1005 = 0. Paid amounts: 6400, 2000, 13000, 0 and 0 respectively.

4. Report 3: Services Ordered During Reservations (JOIN)

```
SELECT so.service_order_id, r.reservation_id, s.service_name,
       so.quantity, s.price, (so.quantity * s.price) AS line_total
FROM ServiceOrders so
JOIN Reservations r ON so.reservation_id = r.reservation_id
JOIN Services s ON so.service_id = s.service_id
ORDER BY so.service_order_id;
```

Purpose: Lists each service request and calculates its line total. Expected line totals: order 701 = 500; 702 = 1000; 703 = 1200; 704 = 900; 705 = 250.

5. Report 4: Reservations Above the Average Room Rate (Subquery)

```
SELECT reservation_id, booked_room_rate
FROM Reservations
WHERE booked_room_rate > (
  SELECT AVG(booked_room_rate) FROM Reservations
)
```

```
ORDER BY booked_room_rate DESC;
```

Purpose: Finds reservations whose booked room rate is higher than the average booked rate. The average is 3580.00, so reservation 1003 (6500.00) is returned.

6. Report 5: Guests Who Have Made a Reservation (EXISTS subquery)

```
SELECT g.guest_id, g.first_name, g.last_name
FROM Guests g
WHERE EXISTS (
  SELECT 1 FROM Reservations r WHERE r.guest_id = g.guest_id
)
ORDER BY g.guest_id;
```

Purpose: Finds guests with at least one reservation. With the sample data, all five guests are returned.

7. Report 6: Service Revenue Estimate by Service

```
SELECT s.service_name, SUM(so.quantity) AS units_ordered,
       SUM(so.quantity * s.price) AS estimated_service_total
FROM Services s
JOIN ServiceOrders so ON s.service_id = so.service_id
GROUP BY s.service_id, s.service_name
ORDER BY estimated_service_total DESC;
```

service_name	units_ordered	estimated_service_total
Spa	1	1200.00
Room Dining	2	1000.00
Airport Pickup	1	900.00
Laundry	3	750.00

Purpose: Estimates the value of ordered services using the current service prices. Historical reporting would be more accurate if the price charged at the time of each order were also stored in ServiceOrders.

8. Conclusion

The reports use joins, grouping, calculated columns and subqueries to answer practical questions about bookings, guests, payments and services. The query results can be checked in MySQL Workbench, and screenshots of actual execution can be added to the final submission.